

P&D CONNECT

Padrões de Projeto (GoF) Aplicados ao Projeto

Análise de aplicabilidade: Singleton, Builder, Factory Method, Abstract Factory, Facade e Adapter

Cliente	AC2 Microbiologia — Núcleo de Pesquisa e Desenvolvimento
Base de referência	TAP, Declaração de Escopo v1.2, Modelo de Classes Preliminar
Data	14/09/2026

1. Objetivo do Documento

Este documento avalia a aplicabilidade de seis padrões de projeto (GoF) já estudados em aula — Singleton, Builder, Factory Method, Abstract Factory, Facade e Adapter — ao contexto real do P&D Connect, com base no Modelo de Classes Preliminar e nos requisitos e riscos já documentados no TAP e na Declaração de Escopo.

2. Resumo da Avaliação

Padrão	Aplicabilidade	Onde se aplica
Builder	Forte	Construção progressiva da Proposta Estruturada pelo Copiloto IA
Facade	Forte	Copiloto IA, Motor de Matching e Motor de Pré-análise
Adapter	Forte	Integração com provedor externo de IA (troca de fornecedor)
Factory Method	Boa	Criação das subclasses de Usuario no cadastro
Abstract Factory	Opcional	Só se houver troca de famílias inteiras de dependências externas
Singleton	Fraca	Gestão de configuração/conexão única com a API de IA

3. Padrões com Aplicabilidade Forte

3.1 Builder — Construção da Proposta Estruturada

Motivação: o TAP já descreve a estruturação da proposta como um processo incremental, não uma criação de uma só vez.

“A IA analisará as informações cadastradas, identificará lacunas e orientará a construção de elementos como problema de pesquisa, hipótese, objetivos, metodologia, resultados esperados, inovação, infraestrutura e recursos necessários. Esse processo será iterativo, permitindo que a proposta seja complementada e reavaliada conforme evolui. (TAP, seção Descrição Técnica da Proposta)”

Aplicação: um PropostaBuilder acumula cada parte da proposta (problema, hipótese, objetivos, metodologia...) conforme o Supervisor responde às perguntas orientadoras do Copiloto IA, e só ao final consolida o objeto PropostaEstruturada completo — mapeando diretamente para os casos de uso UC-05 (Estruturar Proposta) e PB29 a PB41 do Product Backlog.

```
builder = PropostaBuilder(oportunidade)
builder.definir_problema(texto)
builder.definir_hipotese(texto)
builder.definir_objetivos(geral, especificos)
builder.definir_metodologia(texto)
builder.definir_resultados_esperados(texto)
builder.definir_inovacao(texto)
builder.definir_infraestrutura(texto)
proposta = builder.consolidar() # retorna PropostaEstruturada, versao=1
```

3.2 Facade — Simplificação das Três Frentes de Valor

Motivação: o TAP organiza o produto em três frentes centrais (estruturação, matching, pré-análise), cada uma com lógica interna complexa que não deve vazar para as views/controllers do Django.

Aplicação: cada frente ganha uma classe de fachada com um método de entrada único, escondendo prompt engineering, chamadas a serviços externos, parsing de resposta e regras de composição.

```
class CopilotoIAFacade:
    def estruturar(self, oportunidade) -> PropostaEstruturada: ...

class MatchingFacade:
    def executar(self, proposta) -> EquipePotencial: ...

class PreAnaliseFacade:
    def avaliar(self, proposta) -> PreAnaliseMaturidade: ...

# na view/service do Django:
proposta = CopilotoIAFacade().estruturar(oportunidade)
```

Vantagem prática: os métodos de negócio já previstos no Modelo de Classes Preliminar (PropostaEstruturada.consolidar(), EquipePotencial.calcular_lacunas_equipe(), PreAnaliseMaturidade.gerar_relatorio()) passam a ser orquestrados internamente por essas fachadas, mantendo a API do sistema simples e testável.

3.3 Adapter — Independência do Provedor de IA

Motivação: este é o único padrão com respaldo explícito no registro de riscos do TAP.

“Dependência ou indisponibilidade de serviço/modelo externo de IA, caso adotado. Probabilidade: média. Impacto: médio. Resposta: desacoplar a integração e prever tratamento de falhas/substituição do provedor. (TAP, seção Riscos)”

Aplicação: define-se uma interface interna única para o que o sistema precisa da IA, e cada provedor externo ganha um adaptador que traduz essa interface para sua API específica.

```
class IAProviderInterface:
    def gerar_resposta(self, prompt: str) -> str: ...

class OpenAIAdapter(IAProviderInterface):
    def gerar_resposta(self, prompt): ... # traduz para a API da OpenAI

class AnthropicAdapter(IAProviderInterface):
    def gerar_resposta(self, prompt): ... # traduz para a API da Anthropic
```

A CopilototIAFacade conhece apenas IAProviderInterface — trocar de provedor de IA vira uma mudança de configuração, não uma reescrita de código, exatamente como o TAP pede.

4. Demais Padrões Avaliados

4.1 Factory Method — Criação das Subclasses de Usuario (aplicabilidade boa)

Motivação: o Modelo de Classes Preliminar já define herança (Usuario → DemandanteExterno / Pesquisador / Supervisor / Administrador), e o Fechamento de Atores estabelece uma regra de negócio explícita sobre quem pode criar cada tipo.

“Pesquisadores não podem realizar autocadastro; seu cadastro é realizado exclusivamente por Supervisor, após o qual recebem acesso para login. (Fechamento de Atores, RN-A04)”

```
class UsuarioFactory:
    @staticmethod
    def criar(tipo: str, dados: dict) -> Usuario:
        if tipo == "demandante":
            return DemandanteExterno(**dados)
        elif tipo == "pesquisador":
            return Pesquisador(**dados)    # validar cadastrado_por=Supervisor
aqui
        elif tipo == "supervisor":
            return Supervisor(**dados)
        elif tipo == "administrador":
            return Administrador(**dados)
        raise ValueError(f"Tipo de usuário inválido: {tipo}")
```

Centralizar a criação nesse ponto único facilita validar e testar a regra RN-A04 em um só lugar, em vez de espalhar a verificação pelo código.

4.2 Abstract Factory (aplicabilidade opcional — não recomendado para o MVP)

Abstract Factory faz sentido quando é preciso trocar famílias inteiras de dependências relacionadas de uma vez (por exemplo, cliente de IA + parser de resposta + formato de prompt, como um conjunto "família OpenAI" vs. "família Anthropic").

Avaliação: para o escopo atual do MVP, o Adapter isolado já resolve o problema real identificado no TAP (troca de um único provedor de IA). Introduzir Abstract Factory aqui adicionaria uma camada de abstração sem uma necessidade concreta que a justifique — recomendação: mencionar como alternativa considerada e descartada por redundância, não implementar no MVP.

4.3 Singleton (aplicabilidade fraca)

Em Django, models, settings e conexões já se comportam como instância única por natureza do próprio framework — aplicar Singleton manualmente sobre esse comportamento é, em boa parte, redundante.

Um uso legítimo, ainda que de impacto limitado, seria um IAClientManager garantindo que existe apenas uma configuração/conexão ativa com a API de IA durante o tempo de vida da aplicação, evitando reautenticação a cada chamada.

```
class IAClientManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._client = None
        return cls._instance

    def get_client(self):
        if self._client is None:
            self._client = self._inicializar_conexao()
        return self._client
```

Recomendação: citar como padrão avaliado e aplicado de forma pontual, sem apresentá-lo como peça central da arquitetura.

5. Conclusão e Recomendação

Dos seis padrões avaliados, quatro têm aplicação justificada por uma necessidade real e documentada do P&D Connect — Builder, Facade, Adapter e Factory Method — cobrindo, respectivamente, a construção incremental da proposta, a organização das três frentes de valor do produto, a independência de provedor de IA (risco já registrado no TAP) e a criação controlada dos quatro tipos de usuário.

Abstract Factory e Singleton foram avaliados e não são recomendados como peças centrais: o primeiro é redundante frente ao Adapter para o escopo atual do MVP, e o segundo tem utilidade limitada dado que o Django já resolve boa parte do que o Singleton clássico endereça. Ambos podem constar na documentação como padrões considerados, com a devida justificativa de não adoção — isso também demonstra maturidade de decisão de arquitetura.

Próximo passo sugerido: ao iniciar a implementação dos models e services em Django, mapear cada Facade e cada Adapter definidos aqui para módulos concretos do projeto (ex.: `app services/` com `copiloto_ia.py`, `matching.py`, `preanalise.py`, `ia_providers.py`), mantendo a rastreabilidade com este documento.